

CONTENTS

Saving Rules

Listing and Deleting Rules

Generally Useful Rules

Blocking an IP Address

Service: SSH

Service: Web Server

Service: MySQL

Service: PostgreSQL

Service: Mail

Conclusion

// TUTORIAL //

Iptables Essentials: Common Firewall Rules and Commands

Updated on July 9, 2021

Ubuntu Firewall



Mitchell Anicas



Introduction

Iptables is a software firewall for Linux distributions. This cheat sheet-style guide provides a quick reference to iptables commands that will create firewall rules that are useful in common, everyday scenarios. This includes iptables examples of allowing and blocking various services by port, network interface, and source IP address.

How To Use This Guide

- Most of the rules that are described here assume that your iptables is set to **DROP** incoming traffic, through the default input policy, and you want to selectively allow inbound traffic
- Use whichever subsequent sections are applicable to what you are trying to achieve. Most sections are not predicated on any other, so you can use the examples below independently
- Use the Contents menu on the right side of this page (at wide page widths) or your browser's find function to locate the sections you need
- Copy and paste the command-line examples given, substituting the highlighted values with your own

Keep in mind that the order of your rules matter. All of these `iptables` commands use the `-A` option to append the new rule to the end of a chain. If you want to put it somewhere else in the chain, you can use the `-I` option which allows you to specify the position of the new rule (or place it at the beginning of the chain by not specifying a rule number).

Note: When working with firewalls, take care not to lock yourself out of your own server by blocking SSH traffic (port 22, by default). If you lose access due to your firewall settings, you may need to connect to it via a web-based console to fix your access. If you're using DigitalOcean, you can read [our Recovery Console product documentation](#) for more information. Once you are connected via the console, you can change your firewall rules to allow SSH access (or allow all traffic). If your saved firewall rules allow SSH access, another method is to reboot your server.

Remember that you can check your current iptables ruleset with `sudo iptables -S` and `sudo iptables -L`.

Let's take a look at the iptables commands!

Saving Rules

Iptables rules are ephemeral, which means they need to be manually saved for them to persist after a reboot.

On Ubuntu, one way to save iptables rules is to use the `iptables-persistent` package. Install it with `apt` like this:

Copy

```
$ sudo apt install iptables-persistent
```

During the installation, you will be asked if you want to save your current firewall rules.

If you update your firewall rules and want to save the changes, run this command:

Copy

```
$ sudo netfilter-persistent save
```

Other Linux distributions may have alternate ways of making your iptables changes permanent. Please refer to the relevant documentation for more information.

Listing and Deleting Rules

If you want to learn how to list and delete iptables rules, check out this tutorial: [How To List and Delete Iptables Firewall Rules](#).

Generally Useful Rules

This section includes a variety of iptables commands that will create rules that are generally useful on most servers.

Allowing Loopback Connections

The **loopback** interface, also referred to as `lo`, is what a computer uses to forward network connections to itself. For example, if you run `ping localhost` or `ping 127.0.0.1`, your server will ping itself using the loopback. The loopback interface is also used if you configure your application server to connect to a database server with a `localhost` address. As such, you will want to be sure that your firewall is allowing these connections.

To accept all traffic on your loopback interface, run these commands:

Copy

```
$ sudo iptables -A INPUT -i lo -j ACCEPT
$ sudo iptables -A OUTPUT -o lo -j ACCEPT
```

Allowing Established and Related Incoming Connections

As network traffic generally needs to be two-way – incoming and outgoing – to work properly, it is typical to create a firewall rule that allows **established** and **related** incoming traffic, so that the server will allow return traffic for outgoing connections initiated by the server itself. This command will allow that:

Copy

```
$ sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
```

Allowing Established Outgoing Connections

You may want to allow outgoing traffic of all **established** connections, which are typically the response to legitimate incoming connections. This command will allow that:

Copy

```
$ sudo iptables -A OUTPUT -m conntrack --ctstate ESTABLISHED -j ACCEPT
```

Allowing Internal Network to access External

Assuming `eth0` is your external network, and `eth1` is your internal network, this will allow your internal to access the external:

Copy

```
$ sudo iptables -A FORWARD -i eth1 -o eth0 -j ACCEPT
```

Dropping Invalid Packets

Some network traffic packets get marked as **invalid**. Sometimes it can be useful to log this type of packet but often it is fine to drop them. Do so with this command:

Copy

```
$ sudo iptables -A INPUT -m conntrack --ctstate INVALID -j DROP
```

Blocking an IP Address

To block network connections that originate from a specific IP address, `203.0.113.51` for example, run this command:

Copy

```
$ sudo iptables -A INPUT -s 203.0.113.51 -j DROP
```

In this example, `-s 203.0.113.51` specifies a **source** IP address of "203.0.113.51". The source IP address can be specified in any firewall rule, including an **allow** rule.

If you want to **reject** the connection instead, which will respond to the connection request with a "connection refused" error, replace "DROP" with "REJECT" like this:

Copy

```
$ sudo iptables -A INPUT -s 203.0.113.51 -j REJECT
```

Blocking Connections to a Network Interface

To block connections from a specific IP address, e.g. `203.0.113.51`, to a specific network interface, e.g. `eth0`, use this command:

Copy

```
$ iptables -A INPUT -i eth0 -s 203.0.113.51 -j DROP
```

This is the same as the previous example, with the addition of `-i eth0`. The network interface can be specified in any firewall rule, and is a great way to limit the rule to a particular network.

Service: SSH

If you're using a server without a local console, you will probably want to allow incoming SSH connections (port 22) so you can connect to and manage your server. This section covers how to configure your firewall with various SSH-related rules.

Allowing All Incoming SSH

To allow all incoming SSH connections run these commands:

Copy

```
$ sudo iptables -A INPUT -p tcp --dport 22 -m conntrack --ctstate NEW,ESTABLISHED -j ACCEPT
$ sudo iptables -A OUTPUT -p tcp --sport 22 -m conntrack --ctstate ESTABLISHED -j ACCEPT
```

The second command, which allows the outgoing traffic of **established** SSH connections, is only necessary if the `OUTPUT` policy is not set to `ACCEPT`.

Allowing Incoming SSH from Specific IP address or subnet

To allow incoming SSH connections from a specific IP address or subnet, specify the source. For example, if you want to allow the entire `203.0.113.0/24` subnet, run these commands:

Copy

```
$ sudo iptables -A INPUT -p tcp -s 203.0.113.0/24 --dport 22 -m conntrack --ctstate NEW,ESTABLISHED
$ sudo iptables -A OUTPUT -p tcp --sport 22 -m conntrack --ctstate ESTABLISHED -j ACCEPT
```

The second command, which allows the outgoing traffic of **established** SSH connections, is only necessary if the `OUTPUT` policy is not set to `ACCEPT`.

Allowing Outgoing SSH

If your firewall `OUTPUT` policy is not set to `ACCEPT`, and you want to allow outgoing SSH connections—your server initiating an SSH connection to another server—you can run these commands:

Copy

```
$ sudo iptables -A OUTPUT -p tcp --dport 22 -m conntrack --ctstate NEW,ESTABLISHED -j ACCEPT
$ sudo iptables -A INPUT -p tcp --sport 22 -m conntrack --ctstate ESTABLISHED -j ACCEPT
```

Allowing Incoming Rsync from Specific IP Address or Subnet

Rsync, which runs on port 873, can be used to transfer files from one computer to another.

To allow incoming rsync connections from a specific IP address or subnet, specify the source IP address and the destination port. For example, if you want to allow the entire `203.0.113.0/24` subnet to be able to rsync to your server, run these commands:

Copy

```
$ sudo iptables -A INPUT -p tcp -s 203.0.113.0/24 --dport 873 -m conntrack --ctstate NEW,ESTABLISHED
$ sudo iptables -A OUTPUT -p tcp --sport 873 -m conntrack --ctstate ESTABLISHED -j ACCEPT
```

The second command, which allows the outgoing traffic of **established** rsync connections, is only necessary if the `OUTPUT` policy is not set to `ACCEPT`.

Service: Web Server

Web servers, such as Apache and Nginx, typically listen for requests on port 80 and 443 for HTTP and HTTPS connections, respectively. If your default policy for incoming traffic is set to drop or deny, you will want to create rules that will allow your server to respond to those requests.

Allowing All Incoming HTTP

To allow all incoming HTTP (port 80) connections run these commands:

[Copy](#)

```
$ sudo iptables -A INPUT -p tcp --dport 80 -m conntrack --ctstate NEW,ESTABLISHED -j ACCEPT
$ sudo iptables -A OUTPUT -p tcp --sport 80 -m conntrack --ctstate ESTABLISHED -j ACCEPT
```

The second command, which allows the outgoing traffic of **established** HTTP connections, is only necessary if the `OUTPUT` policy is not set to `ACCEPT`.

Allowing All Incoming HTTPS

To allow all incoming HTTPS (port 443) connections run these commands:

[Copy](#)

```
$ sudo iptables -A INPUT -p tcp --dport 443 -m conntrack --ctstate NEW,ESTABLISHED -j ACCEPT
$ sudo iptables -A OUTPUT -p tcp --sport 443 -m conntrack --ctstate ESTABLISHED -j ACCEPT
```

The second command, which allows the outgoing traffic of **established** HTTP connections, is only necessary if the `OUTPUT` policy is not set to `ACCEPT`.

Allowing All Incoming HTTP and HTTPS

If you want to allow both HTTP and HTTPS traffic, you can use the **multiport** module to create a rule that allows both ports. To allow all incoming HTTP and HTTPS (port 443) connections run these commands:

[Copy](#)

```
o iptables -A INPUT -p tcp -m multiport --dports 80,443 -m conntrack --ctstate NEW,ESTABLISHED -j ACCEPT
o iptables -A OUTPUT -p tcp -m multiport --dports 80,443 -m conntrack --ctstate ESTABLISHED -j ACCEPT
```

The second command, which allows the outgoing traffic of **established** HTTP and HTTPS connections, is only necessary if the `OUTPUT` policy is not set to `ACCEPT`.

Service: MySQL

MySQL listens for client connections on port 3306. If your MySQL database server is being used by a client on a remote server, you need to be sure to allow that traffic.

Allowing MySQL from Specific IP Address or Subnet

To allow incoming MySQL connections from a specific IP address or subnet, specify the source. For

Products	>
Solutions	>
Developers	>
Partners	>
Pricing	

[Log in](#)[Sign up](#)[Blog](#)[Docs](#)

[Get Support](#)[Contact Sales](#)[Tutorials](#)[Questions](#)[Learning Paths](#)[For Businesses](#)[Product Docs](#)[Social Impact](#)[Search Commun](#)

Service: PostgreSQL

PostgreSQL listens for client connections on port 5432. If your PostgreSQL database server is being used by a client on a remote server, you need to be sure to allow that traffic.

PostgreSQL from Specific IP Address or Subnet

To allow incoming PostgreSQL connections from a specific IP address or subnet, specify the source. For example, if you want to allow the entire `203.0.113.0/24` subnet, run these commands:

[Copy](#)

```
$ sudo iptables -A INPUT -p tcp -s 203.0.113.0/24 --dport 5432 -m conntrack --ctstate NEW,ESTABLISHED
$ sudo iptables -A OUTPUT -p tcp --sport 5432 -m conntrack --ctstate ESTABLISHED -j ACCEPT
```

The second command, which allows the outgoing traffic of **established** PostgreSQL connections, is only necessary if the `OUTPUT` policy is not set to `ACCEPT`.

Allowing PostgreSQL to Specific Network Interface

To allow PostgreSQL connections to a specific network interface—say you have a private network interface `eth1`, for example—use these commands:

[Copy](#)

```
$ sudo iptables -A INPUT -i eth1 -p tcp --dport 5432 -m conntrack --ctstate NEW,ESTABLISHED -j ACCEPT
$ sudo iptables -A OUTPUT -o eth1 -p tcp --sport 5432 -m conntrack --ctstate ESTABLISHED -j ACCEPT
```

The second command, which allows the outgoing traffic of **established** PostgreSQL connections, is only necessary if the `OUTPUT` policy is not set to `ACCEPT`.

Service: Mail

Mail servers, such as Sendmail and Postfix, listen on a variety of ports depending on the protocols being used for mail delivery. If you are running a mail server, determine which protocols you are using and allow the appropriate types of traffic. We will also show you how to create a rule to block outgoing SMTP mail.

Blocking Outgoing SMTP Mail

If your server shouldn't be sending outgoing mail, you may want to block that kind of traffic. To block outgoing SMTP mail, which uses port 25, run this command:

[Copy](#)

```
$ sudo iptables -A OUTPUT -p tcp --dport 25 -j REJECT
```

This configures iptables to **reject** all outgoing traffic on port 25. If you need to reject a different service by its port number, instead of port 25, substitute that port number for the `25` above.

Allowing All Incoming SMTP

To allow your server to respond to SMTP connections on port 25, run these commands:

Copy

```
$ sudo iptables -A INPUT -p tcp --dport 25 -m conntrack --ctstate NEW,ESTABLISHED -j ACCEPT
$ sudo iptables -A OUTPUT -p tcp --sport 25 -m conntrack --ctstate ESTABLISHED -j ACCEPT
```

The second command, which allows the outgoing traffic of **established** SMTP connections, is only necessary if the `OUTPUT` policy is not set to `ACCEPT`.

Allowing All Incoming IMAP

To allow your server to respond to IMAP connections, port 143, run these commands:

Copy

```
$ sudo iptables -A INPUT -p tcp --dport 143 -m conntrack --ctstate NEW,ESTABLISHED -j ACCEPT
$ sudo iptables -A OUTPUT -p tcp --sport 143 -m conntrack --ctstate ESTABLISHED -j ACCEPT
```

The second command, which allows the outgoing traffic of **established** IMAP connections, is only necessary if the `OUTPUT` policy is not set to `ACCEPT`.

Allowing All Incoming IMAPS

To allow your server to respond to IMAPS connections, port 993, run these commands:

Copy

```
$ sudo iptables -A INPUT -p tcp --dport 993 -m conntrack --ctstate NEW,ESTABLISHED -j ACCEPT
$ sudo iptables -A OUTPUT -p tcp --sport 993 -m conntrack --ctstate ESTABLISHED -j ACCEPT
```

The second command, which allows the outgoing traffic of **established** IMAPS connections, is only necessary if the `OUTPUT` policy is not set to `ACCEPT`.

Allowing All Incoming POP3

To allow your server to respond to POP3 connections, port 110, run these commands:

Copy

```
$ sudo iptables -A INPUT -p tcp --dport 110 -m conntrack --ctstate NEW,ESTABLISHED -j ACCEPT
$ sudo iptables -A OUTPUT -p tcp --sport 110 -m conntrack --ctstate ESTABLISHED -j ACCEPT
```

The second command, which allows the outgoing traffic of **established** POP3 connections, is only necessary if the `OUTPUT` policy is not set to `ACCEPT`.

Allowing All Incoming POP3S

To allow your server to respond to POP3S connections, port 995, run these commands:

Copy

```
$ sudo iptables -A INPUT -p tcp --dport 995 -m conntrack --ctstate NEW,ESTABLISHED -j ACCEPT
$ sudo iptables -A OUTPUT -p tcp --sport 995 -m conntrack --ctstate ESTABLISHED -j ACCEPT
```

The second command, which allows the outgoing traffic of **established** POP3S connections, is only necessary if the `OUTPUT` policy is not set to `ACCEPT`.

Conclusion

That should cover many of the commands that are commonly used when configuring an iptables firewall. Of course, iptables is a very flexible tool so feel free to mix and match the commands with different options to match your specific needs if they aren't covered here.

If you're looking for help determining how your firewall should be set up, check out this tutorial: [How To Choose an Effective Firewall Policy to Secure your Servers](#).

Easily secure your infrastructure and define what services are visible on your servers using DigitalOcean Cloud Firewalls. Our Cloud Firewalls are free and perfect for staging and production deployments.

[Learn more here →](#)

About the authors



[Mitchell Anicas](#) Author

Still looking for an answer?

[Ask a question](#)

[Search for more help](#)

Was this helpful?

[Yes](#)

[No](#)



Comments

10 Comments

B *I* U



Leave a comment...

This textbox defaults to using **Markdown** to format your answer.

You can type `!ref` in this text area to quickly search our full set of tutorials, documentation & marketplace offerings and insert the link!

[Sign In or Sign Up to Comment](#)

[Tauseef](#) • August 12, 2015



A very helping tutorial and very well explained. Thanks a lot :)

[Reply](#)

lendog • February 18, 2016

Hi, great article - well written and easy to follow as always. Small note - when i was testing out the rules for POP3,

```
sudo iptables -A INPUT -i -p tcp --dport 110 -m conntrack --ctstate NEW,ESTABLISHED -j ACCEPT
```

i encountered the error

"Bad argument `tcp' "

The problem seems to lie with the "-i" after "INPUT". You might want to double check that command - what worked for me is

```
sudo iptables -A INPUT -p tcp --dport 110 -m conntrack --ctstate NEW,ESTABLISHED -j ACCEPT
```

[Reply](#)

indigochild • December 31, 2015

Hey, correct me if I'm wrong, but in the section 'Allow All Incoming HTTP and HTTPS', shouldn't the second firewall rule be:

```
sudo iptables -A OUTPUT -p tcp -m multiport --sports 80,443 -m conntrack --ctstate ESTABLISHED
```

as opposed to

```
sudo iptables -A OUTPUT -p tcp -m multiport --dports 80,443 -m conntrack --ctstate ESTABLISHED
```

?

[Show replies](#) [Reply](#)

4e3b97994aa14af2be3faa0a0b3ed8 • November 23, 2015

Thanks for this really good tutorial! Question: If I block ALL ports except ssh and 80, and even then restrict 80 to only allow incoming connections from Cloudflare's range, I know it'll work. But will it interfere with any of DigitalOcean's management tools like backups and monitoring? Or do they operate at a lower, more direct level?

By the way, I think it would be useful to add to the tutorial the ability to block a range. For example, as I understand it:

Block 15.15.x sudo iptables -A INPUT -s 15.15.0/24 -j DROP

Block 15.15.x.x sudo iptables -A INPUT -s 15.15.0.0/16 -j DROP

Block entire 15.x.x.x (use with caution!) sudo iptables -A INPUT -s 15.0.0.0/8 -j DROP

Someone correct me if I'm wrong, but I've been using /16 to block ranges of baddies with success in the past!

[Show replies](#) ▾ [Reply](#)

Trustyama • October 30, 2015 ^

I appreciate this site. You make a great job ! Thanks !!!

[Reply](#)

YurkshireLad • May 30, 2020 ^

Great article. How would you allow outgoing SMTP requests? Thanks.

[Reply](#)

Abdo • April 9, 2020 ^

it doesn't work, It is not working. The ports still closed, port 25 is still closed.

[Reply](#)

superNUTNUT1 • May 28, 2019 ^

thank you for your tutorial thanks to you I could create this code

```
#!/bin/bash

### NAT ###
iptables -F && iptables -t nat -F
iptables -t nat -A PREROUTING -d 10.1.31.116/24 -p UDP --dport 53 -j DNAT --to-destination 10.1.31.116
iptables -t nat -A PREROUTING -d 10.1.31.116/24 -p TCP --dport 80 -j DNAT --to-destination 10.1.31.116
iptables -t nat -A POSTROUTING -s 10.101.10.128/25 -o enp0s9 -j SNAT --to-source 10.1.31.116

### INPUT ###
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
iptables -A INPUT -i lo -j ACCEPT
iptables -A INPUT -j REJECT

### OUTPUT ###
iptables -A OUTPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
iptables -A OUTPUT -o lo -j ACCEPT
iptables -A OUTPUT -j REJECT

### FORWARD ###
iptables -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

#DMZ peut faire des requetes DNS
iptables -A FORWARD -s 10.101.10.0/25 -p udp --dport 53 -d 10.101.10.130 -j ACCEPT

#Local vers DMZ et internet
iptables -A FORWARD -s 10.101.10.128/25 -d 10.101.10.0/25 -j ACCEPT
```

```
iptables -A FORWARD -s 10.101.10.128/25 -o enp0s9 -j ACCEPT

# connexion internet vers DMZ
iptables -A FORWARD -i enp0s9 -d 10.101.10.130 -p udp --dport 53 -j ACCEPT
iptables -A FORWARD -i enp0s9 -d 10.101.10.20 -p tcp --dport 80 -j ACCEPT
iptables -A FORWARD -j REJECT

iptables -A FORWARD -p icmp --icmp-type 0 -j ACCEPT #request
iptables -A FORWARD -p icmp --icmp-type 8 -j ACCEPT #reply
```

[Reply](#)

[zackinma](#) • April 2, 2019



Awesome.

[Reply](#)

[ainurvaliev](#) • March 28, 2019



it works! Thank you!

[Reply](#)

Load more comments



This work is licensed under a Creative Commons Attribution-NonCommercial- ShareAlike 4.0 International License.

Try DigitalOcean for free

Click below to sign up and get **\$200 of credit** to try our products over 60 days!

[Sign up](#)

Popular Topics

[Ubuntu](#)

[Linux Basics](#)

[JavaScript](#)

[Python](#)

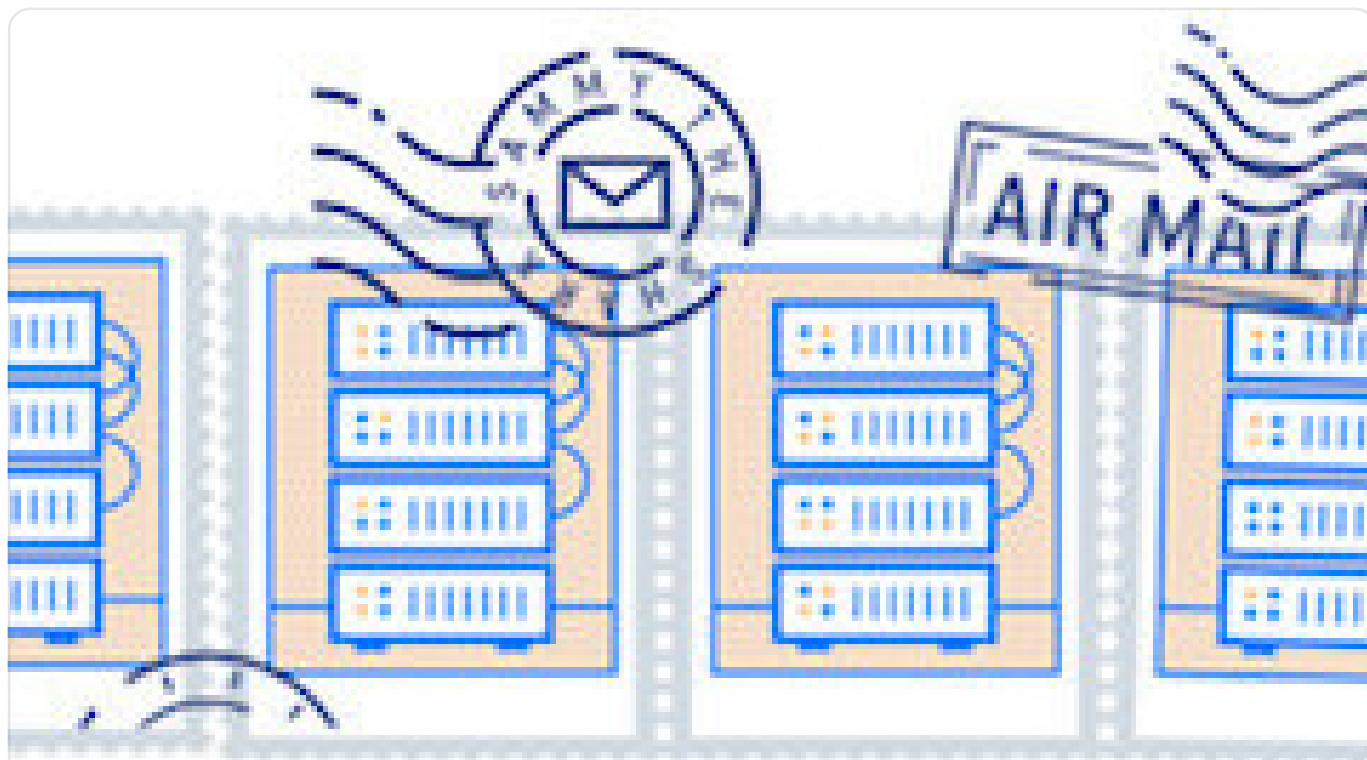
[MySQL](#)

[Docker](#)

Kubernetes

[All tutorials →](#)

[Talk to an expert →](#)



Get our biweekly newsletter

Sign up for Infrastructure as a Newsletter.

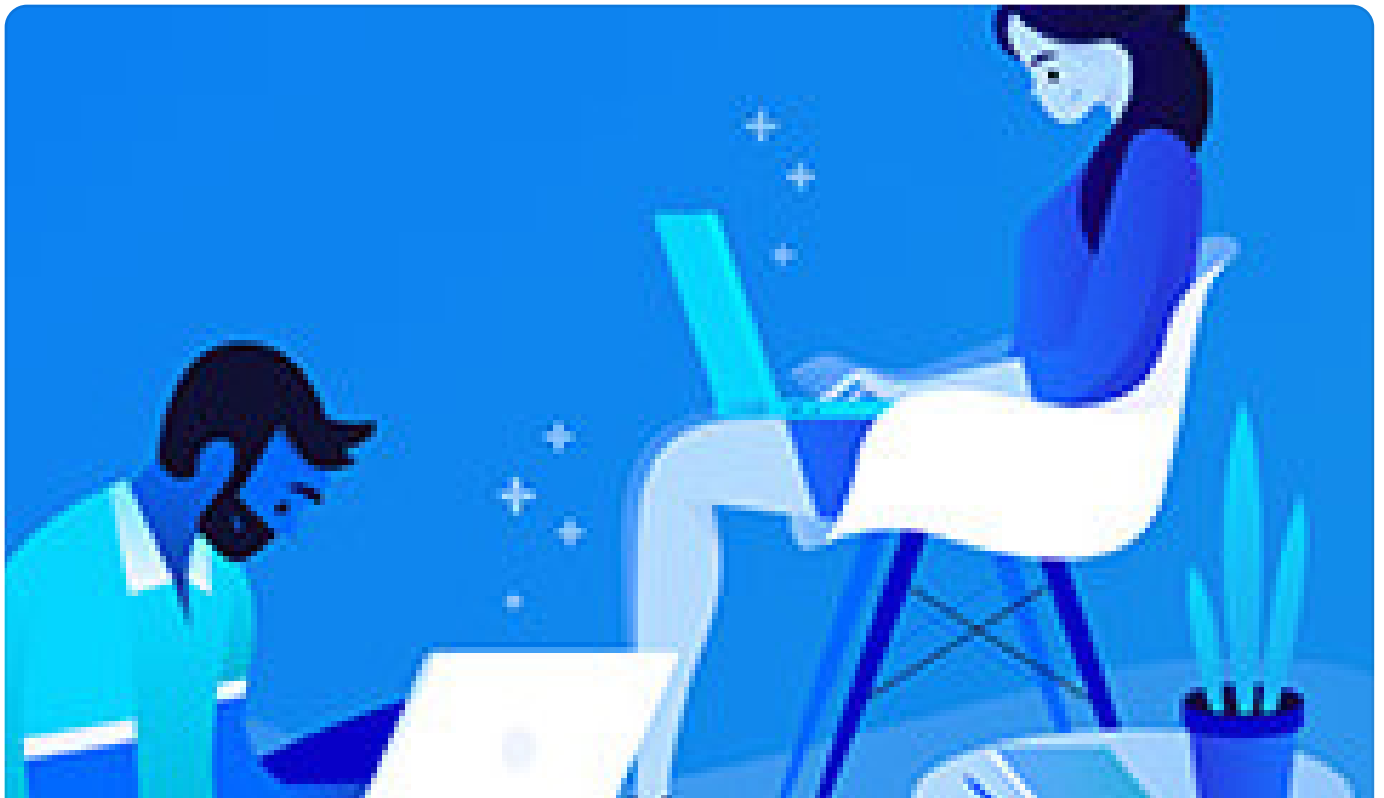
[Sign up →](#)



Hollie's Hub for Good

Working on improving health and education, reducing inequality, and spurring economic growth? We'd like to help.

[Learn more →](#)



Become a contributor

Get paid to write technical tutorials and select a tech-focused charity to receive a matching donation.

[Learn more →](#)

Featured Tutorials

[Kubernetes Course](#) [Learn Python 3](#) [Machine Learning in Python](#) [Getting started with Go](#) [Intro to Kubernetes](#)

DigitalOcean Products

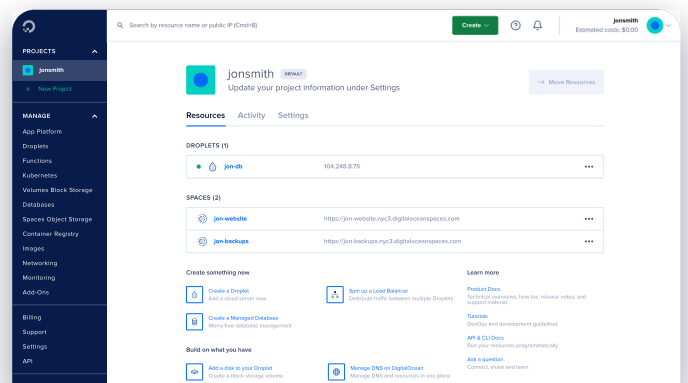
[App Platform](#) [Virtual Machines](#) [Managed Databases](#) [Managed Kubernetes](#) [Block Storage](#) [Object Storage](#)

[Marketplace](#) [VPC](#) [Load Balancers](#)

Welcome to the developer cloud

DigitalOcean makes it simple to launch in the cloud and scale up as you grow — whether you're running one virtual machine or ten thousand.

[Learn more](#)



Company

Products

Resources

Solutions

Contact



© 2024 DigitalOcean, LLC. [Sitemap](#).

